

سیستم بکاپ لینوکس

بکاپ کامل و افزایشی، انتقال امن با SSH و بازیابی مطمئن،
از ایده تا اجرا

cron

SHA-256

SSH / SCP

tar

Bash

فهرست

۱. زیربنا: پروژه چطور ساخته شده ۳
۲. پیش‌نیازها ۶
۳. نحوه استفاده، قدم به قدم و با مثال ۷
۴. خطاها و عیب‌یابی ۱۱

مسئله چیست؟

اگر دیسک ماشین خراب شود، فایل‌ها پاک شوند یا بدافزار به آن‌ها برسد، همه چیز از بین می‌رود. بکاپی که روی همان ماشین بماند در همان لحظه از بین می‌رود. پس یک کپی سالم باید جای دیگری باشد.

راه حل در یک نگاه

کار اول

بسیار

از پوشه‌ی داده یک آرشيو فشرده بگیر، کامل یا فقط تغییرات

کار دوم

بفراست

آن را امن به سرور بده و صحتش را آنجا دوباره بسنج

کار سوم

برگردان

هر وقت لازم شد، داده را مطمئن و بدون خطر بازیابی کن

هدف این راهنما

بعد از خواندن آن باید بتوانید توضیح دهید هر قطعه چه کاری می‌کند، چرا آنجاست و چطور از آن استفاده می‌شود.

مسیر یک بکاپ از Debian تا Kali

۱. ساخت

tar + sha256

فشرده سازی داده و ساخت اثر انگشت فایل

۲. ارسال

scp over SSH

با کلید Ed25519 و بدون رمز، مناسب cron

۳. قرنطینه

incoming/

فایل تازه رسیده، هنوز معتبر حساب نمی شود

۴. ذخیره

store/

فقط بعد از تأیید checksum توسط خود سرور

یک فرمان به نام `run.sh` این مراحل را پشت سر هم اجرا می کند و بعد بکاپ های خیلی قدیمی را پاک می کند. هر مرحله فقط وقتی شروع می شود که قبلی موفق بوده باشد.

مرحله	اگر شکست بخورد
<code>backup.sh</code>	ارسال شروع نمی شود و کد خروج ۱۰ می دهد
<code>transfer.sh</code>	بکاپ سالم و محلی می ماند و اجرای بعدی دوباره تلاش می کند (کد ۲۰ یا ۲۱)
<code>prune.sh</code>	هیچ بکاپی از بین نمی رود و فقط پاکسازی انجام نمی شود (کد ۳۰)

چرا یک فرمان واحد؟

اگر بکاپ ساعت ۲ و ارسال ساعت ۲:۳۰ زمان بندی شود و بکاپ ۳۱ دقیقه طول بکشد، فایل ناقص فرستاده می شود. `run.sh` این مشکل را از ریشه حذف می کند.

```
linux-backup-project/
├─ bin/      run.sh  backup.sh  transfer.sh  restore.sh  prune.sh
├─ lib/      common.sh      # تنظیمات، لاگ، قفل، کد خروج
├─ config/   backup.conf    # تنها فایلی که ویرایش می‌کنید
├─ server/   backup-gate.sh  # روی سرور نصب می‌شود
├─ tools/    report.py      # گزارش و بررسی سلامت
├─ tests/    selftest.sh   # تست خودکار
└─ data/     backups/ state/ logs/  # پوشه‌های زمان اجرا
```

وظیفه‌ی هر اسکریپت

فایل	کارش به زبان ساده
run.sh	مدیر هماهنگ‌کننده: بکاپ، سپس ارسال، سپس پاکسازی. این یکی در cron می‌رود
backup.sh	ساخت آرشیو full یا incremental یا auto و فایل sha256. کنارش
transfer.sh	آپلود آرشیوهای ارسال نشده و گرفتن تأیید سرور
restore.sh	نمایش لیست بکاپ‌ها و بازیابی داده به یک پوشه‌ی جدید
prune.sh	نگه‌داشتن چند «مجموعه»ی آخر و حذف قدیمی‌ها
backup-gate.sh	نگهبان سرور: فقط سه درخواست مجاز را می‌پذیرد
report.py	(پایتون) فهرست بکاپ‌ها و بررسی checksum

سه مفهوم کلیدی

۱. بکاپ کامل و افزایشی

Full همه چیز را می‌گیرد. **Incremental** فقط فایل‌های جدید یا تغییرکرده نسبت به دفعه‌ی قبل را می‌گیرد و حذف‌شده‌ها را هم ثبت می‌کند. یک **مجموعه (Set)** یعنی یک **Full** و **Incremental**‌های بعد از آن.

← دو مجموعه (قدیمی‌ترین در راست)



برای بازیابی، هم **Full** و هم همه‌ی **Incremental**‌های بعدش باید سالم باشند و به ترتیب اعمال شوند. به همین دلیل مجموعه‌ها همیشه کامل پاک می‌شوند.

۲. Checksum یا اثر انگشت فایل

SHA-256 عددی است که از محتوای فایل ساخته می‌شود. اگر حتی یک بایت عوض شود، عدد عوض می‌شود. پروژه چهار بار آن را می‌سنجد:



۳. کلید محدود و نگهبان (Gate)

کلید SSH رمز ندارد تا cron بتواند استفاده کند. پس اگر دزدیده شود باید تقریباً بی‌فایده باشد. این اصل «حداقل دسترسی» است.

مجاز	ممنوع
ping برای تست اتصال	باز کردن شل و اجرای دستور
آپلود فایل فقط داخل incoming/	خواندن، لیست یا دانلود بکاپ‌ها
verify برای تأیید checksum	حذف یا بازنویسی بکاپ ذخیره‌شده

چرا فایل اول به incoming می‌رود؟

اگر کسی کلید را بدزد و فایل مخرب با اسم یک بکاپ قدیمی بفرستد، آن فایل فقط در قرنطینه می‌ماند. فایلی که از قبل در store/ هست هرگز بازنویسی نمی‌شود و فایل تازه فقط با checksum درست وارد store/ می‌شود.

پیش‌نیازها

۲

قبل از شروع چه چیزهایی باید آماده باشد

مورد	Debian کلاينت · 192.168.206.129	Kali سرور · 192.168.206.131
نقش	اسکرپت‌ها و داده اینجاست و بکاپ ساخته می‌شود	مقصد؛ بکاپ‌های رسیده اینجا ذخیره می‌شوند
نرم‌افزار	bash tar gzip sha256sum cron openssh-client python3	openssh-server scp sha256sum
کاربر	کاربر عادی (youngev)	کاربر اختصاصی bkpuser
پوشه	~/linux-backup-project	/srv/backups/myhost/incoming /srv/backups/myhost/store
فایل ویژه	config/backup.conf و کلید ~/.ssh/backup_ed25519	/usr/local/bin/backup-gate.sh و authorized_keys

چک‌لیست سریع

(۱) دو ماشین همدیگر را ببینند. روی Debian اجرا کنید و باید ۰٪ packet loss ببینید:

```
ping -c 2 192.168.206.131
```

(۲) SSH روی Kali روشن باشد:

```
sudo systemctl status ssh --no-pager # باید فعال باشد  
# اگر نصب نبود  
sudo apt install openssh-server && sudo systemctl enable --now ssh
```

(۳) نام کاربر بکاپ backup نباشد. این نام در Debian و Kali از قبل یک حساب سیستمی است. ما bkpuser را ساختیم.

نحوه استفاده

راهاندازی یکبار، سپس استفاده‌ی روزانه

۳

قدم ۰۱ آماده‌سازی سرور Kali

```
sudo adduser bkpuser  
sudo mkdir -p /srv/backups/myhost/incoming /srv/backups/myhost/store  
sudo chown -R bkpuser:bkpuser /srv/backups/myhost
```

قدم ۰۲ فرستادن نگهبان به سرور

روی Debian :

```
cd ~/linux-backup-project  
scp server/backup-gate.sh youngev@192.168.206.131:/tmp/
```

سپس روی Kali ، به‌عنوان مالک root تا کاربر بکاپ نتواند آن را ویرایش کند:

```
sudo install -o root -g root -m 755 /tmp/backup-gate.sh /usr/local/bin/backup-gate.sh
```

قدم ۰۳ ساخت کلید Debian

```
ssh-keygen -t ed25519 -f ~/.ssh/backup_ed25519 -C "backup@$(hostname)" -N ""  
chmod 600 ~/.ssh/backup_ed25519  
scp ~/.ssh/backup_ed25519.pub youngev@192.168.206.131:/tmp/
```

فقط فایل `.pub` را بفرستید

فایل بدون `.pub` کلید خصوصی است و هرگز از ماشین خودش بیرون نمی‌رود.

قدم ۴ · فعال سازی کلید، همراه با محدودیت **Kali**

```
sudo mkdir -p /home/bkpuser/.ssh
echo "restrict,command=\"/usr/local/bin/backup-gate.sh\",from=\"192.168.206.129\" $(cat /
tmp/backup_ed25519.pub)" \
| sudo tee /home/bkpuser/.ssh/authorized_keys
sudo chown -R bkpuser:bkpuser /home/bkpuser/.ssh
sudo chmod 700 /home/bkpuser/.ssh && sudo chmod 600 /home/bkpuser/.ssh/authorized_keys
sudo usermod -p '*' bkpuser # رمز را غیرفعال کن؛ از این به بعد فقط کلید
```

گزینه	معنی
restrict	همه‌ی امکانات اضافه را خاموش می‌کند: ترمینال، فوروارد پورت، X11
command=...	هر چه کلاینت بخواهد، فقط همین اسکریپت اجرا می‌شود
from=...	کلید فقط از IP کلاینت کار می‌کند (اختیاری)

قدم ۵ · تست محدودیت‌ها **Debian**

```
ssh -i ~/.ssh/backup_ed25519 bkpuser@192.168.206.131 ping
# → pong
ssh -i ~/.ssh/backup_ed25519 bkpuser@192.168.206.131 ls /
# → backup-gate: command not allowed
```

اگر `pong` گرفتید و `ls` رد شد، یعنی قفل درست کار می‌کند.

قدم ۶ · تنظیمات پروژه **Debian**

```
cd ~/linux-backup-project
cp config/backup.conf.example config/backup.conf
sed -i 's|^REMOTE_USER=.*|REMOTE_USER="bkpuser"|; s|^REMOTE_HOST=.*|
REMOTE_HOST="192.168.206.131"|' config/backup.conf
```

بقیه‌ی مقدارها (پورت ۲۲، مسیر کلید، نگاه‌داری ۴ مجموعه) پیش‌فرض‌اند و درست هستند.

Debian اولین اجرا و استفاده‌ی روزانه

```
echo "hello backup" > data/a.txt
bin/run.sh full # بکاپ کامل + ارسال + پاکسازی
```

روز بعد، پس از تغییر فایل‌ها، فقط تغییرات گرفته می‌شود. خروجی واقعی یک اجرا:

```
[backup ] START type=incremental file=daily_2026-10-09_12-46-47_incremental.tar.gz
[backup ] RESULT status=SUCCESS type=incremental size=206B duration=0s
[transfer] Uploading daily_2026-10-09_12-46-47_incremental.tar.gz (206B)
[transfer] RESULT status=SUCCESS stage=remote-verify (verified by server)
[prune ] RESULT status=SUCCESS prune kept_sets=2 deleted_archives=0
[run ] Pipeline finished OK
```

دستور	کار
<code>bin/run.sh auto</code>	هر هفته یک Full، بقیه‌ی روزها Incremental (برای cron)
<code>bin/run.sh full</code>	اجبار به بکاپ کامل
<code>bin/restore.sh --list</code>	دیدن همه‌ی بکاپ‌ها به صورت مجموعه
<code>python3 tools/report.py</code>	بررسی سلامت checksumها
<code>bin/prune.sh --dry-run</code>	فقط نشان بده چه چیزی پاک می‌شد
<code>tail logs/backup.log</code>	دیدن لاگ

اجرای خودکار با cron

```
crontab -e
# هر شب ساعت ۲
0 2 * * * /home/youngev/linux-backup-project/bin/run.sh auto >> /home/youngev/linux-backup-project/logs/cron.log 2>&1
```

فقط یک خط. ترتیب مراحل را خود `run.sh` رعایت می‌کند.

```
bin/restore.sh --list           # چه چیزی هست؟
bin/restore.sh --to /tmp/restored # آخرین وضعیت
bin/restore.sh --to /tmp/restored --backup 2026-10-09_12-43 # یک نقطه‌ی زمانی
bin/restore.sh --to /tmp/restored --dry-run           # فقط برنامه را نشان بده
diff -r data /tmp/restored/data && echo "restore OK"
```

اگر خروجی `diff` خالی بود و `restore OK` دیدید، داده‌ی بازیابی‌شده دقیقاً با داده‌ی فعلی یکی است.

پشت صحنه چه اتفاقی می‌افتد؟

#	مرحله
۱	آرشیو هدف پیدا می‌شود؛ اگر Incremental باشد، Full قبلی و همه‌ی Incremental‌های بین‌شان هم انتخاب می‌شوند
۲	پیش از هر کاری: کامل بودن زنجیره، SHA-256 و gzip هر آرشیو بررسی می‌شود
۳	مقصد باید خالی باشد؛ هیچ فایل‌ی بازنویسی نمی‌شود
۴	استخراج در پوشه‌ی موقت و به ترتیب Incremental ← Full‌ها
۵	فقط اگر همه چیز موفق بود، پوشه‌ی موقت به مقصد منتقل می‌شود؛ وگرنه پاک می‌شود و مقصد دست‌نخورده می‌ماند

بازیابی از روی سرور

کلید بکاپ عمده‌اً فقط «نوشتن» می‌تواند، پس نمی‌تواند چیزی دانلود کند. برای بازیابی با حساب خودتان وارد Kali شوید و آرشیوها را از `store/` بردارید، سپس `restore.sh --dir` را به آن پوشه بدهید.

خطاها و عیب‌یابی

کد خروج می‌گوید دقیقاً چه چیزی خراب شد

کد	معنی	اسکرپت
0	موفق	همه
3	تنظیمات ناقص یا اشتباه	همه
4	کار دیگری در حال اجراست (قفل)	همه
10	ساخت بکاپ شکست خورد؛ ارسال انجام نمی‌شود	backup / run
20	ارسال شکست خورد: شبکه، SSH یا scp؛ بار بعد تکرار می‌شود	transfer / run
21	checksum مطابقت نداشت، محلی یا روی سرور	transfer / run
30	پاکسازی بکاپ‌های قدیمی شکست خورد	prune / run
40	بازیابی پیش از دست‌زدن به چیزی رد شد	restore
41	استخراج شکست خورد؛ فایل‌های موقت پاک شدند	restore

برای پیدا کردن همه‌ی خطاها در لاگ: `grep 'status=FAILED' logs/backup.log`

مشکلات رایج

علامت	بررسی
Permission denied (publickey)	مسیر SSH_KEY؛ خط <code>authorized_keys</code> باید یک خط واحد باشد؛ مجوزهای ۶۰۰ و ۷۰۰
Host key verification failed	یک بار دستی <code>ssh ... ping</code> بزنید و fingerprint را تأیید کنید
کد خروج 20 ولی SSH دستی کار می‌کند	نصب بودن <code>backup-gate.sh</code> و مسیر <code>command=</code> در <code>authorized_keys</code>
در cron کار نمی‌کند ولی دستی چرا	مسیرها مطلق باشند و cron با همان کاربری باشد که کلید را دارد

محدودیت‌های صادقانه‌ی پروژه

بکاپ‌ها روی سرور رمزنگاری نشده‌اند. حذف خودکار بکاپ‌های قدیمی روی سرور هنوز نیست (فقط محلی).
رابط گرافیکی و هشدار خطا برای مراحل بعد برنامه‌ریزی شده‌اند.